

untangle::actuator

Generated by Doxygen 1.18.0

1 actuator	1
2 Topic Index	3
2.1 Topics	3
3 Namespace Index	5
3.1 Namespace List	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Topic Documentation	11
6.1 namespace untangle: functions	11
6.1.1 Detailed Description	11
6.1.2 Function Documentation	11
6.1.2.1 connect() [1/4]	11
6.1.2.2 connect() [2/4]	13
6.1.2.3 connect() [3/4]	13
6.1.2.4 connect() [4/4]	14
6.1.2.5 bind() [1/2]	15
6.1.2.6 bind() [2/2]	15
7 Namespace Documentation	17
7.1 untangle Namespace Reference	17
7.1.1 Detailed Description	18
7.1.2 Function Documentation	18
7.1.2.1 last_arg()	18
7.1.2.2 connect_one()	18
7.1.2.3 connect_named_one()	19
8 Class Documentation	21
8.1 untangle::actuator< action_t > Struct Template Reference	21
8.1.1 Detailed Description	22
8.1.2 Member Typedef Documentation	23
8.1.2.1 actions_t	23
8.1.2.2 results_t	24
8.1.2.3 errors_t	24
8.1.3 Constructor & Destructor Documentation	24
8.1.3.1 actuator()	24
8.1.4 Member Function Documentation	24
8.1.4.1 operator=()	24
8.1.4.2 type()	25
8.1.4.3 reset()	25

8.1.4.4	copy_from()	25
8.1.4.5	release_owned()	26
8.1.4.6	invoke_callback()	26
8.1.4.7	operator>()	27
8.1.4.8	invoke_action()	28
8.1.4.9	add() [1/4]	28
8.1.4.10	add() [2/4]	29
8.1.4.11	add() [3/4]	29
8.1.4.12	add() [4/4]	30
8.1.4.13	remove() [1/2]	30
8.1.4.14	remove() [2/2]	31
8.1.4.15	is_connected()	31
8.1.4.16	has_action()	31
8.1.5	Member Data Documentation	32
8.1.5.1	owned	32
8.2	untangle::invalid_action Struct Reference	32
8.2.1	Detailed Description	32
8.2.2	Constructor & Destructor Documentation	32
8.2.2.1	invalid_action()	32
8.2.3	Member Function Documentation	33
8.2.3.1	what()	33
9	File Documentation	35
9.1	actuator.hpp	35
	Index	41

Chapter 1

actuator

An actuator is a generic callable that may invoke a list of actions(callables of type `std::function<...>`).

Its goal is to provide a light and simple mechanism to connect components that want to communicate with each other. It works similar as the "signal and slots" mechanisms from other frameworks, but much simplified.

Modern programming languages do not offer class intrinsic interfaces that would improve significantly how objects are connected and how they communicate. Instead, languages, like C++, are using external interfaces, that are not only cluttering the code, but they don't feel "natural" as they are opposed to what an interface is in reality. Basically the objects and interfaces are not separated, and the interface is intrinsic to the object(think on the cogs in a mechanism). A generic callable is an attempt to mitigate this problem. `async` implementation is such use case.

An actuator provides the results of all actions in a vector, in the same order as the actions were added.

Example: how to use actuator instead of polymorphism

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle)
{
    for (const auto& s : shapes)
    {
        s->rotate(angle);
    }
}

std::shared_ptr<triangle> t(new triangle);
std::shared_ptr<circle> c(new circle);
std::shared_ptr<square> s(new square);

// using polymorphism
std::vector<shape*> shapes;
shapes.push_back(t.get());
shapes.push_back(c.get());
shapes.push_back(s.get());

rotate_shapes(shapes, 10);

// using actuator
auto action1 = untangle::bind(t, &triangle::rotate);
auto action2 = untangle::bind(c, &circle::rotate);
auto action3 = untangle::bind(s, &square::rotate);
auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

Anonymous lambdas as actions

An actuator normally does not own its actions: it stores pointers to `std::function` objects the caller keeps alive. `add()` also has an overload that takes an action by value, so an anonymous lambda can be an action with no named variable to keep around, and returns a handle to pass to `remove()`:

```
untangle::actuator<std::function<int(int)>> actuator_scale;
auto* handle = actuator_scale.add([](int v) { return v * 2; });
actuator_scale.add("triple", [](int v) { return v * 3; });
```

`connect()` accepts them too, but it deduces the action type from its arguments, and a lambda has its own closure type – it is not a `std::function` until something converts it. A call made only of anonymous lambdas therefore has nothing to deduce from, and the signature has to be named on `connect()` itself:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    [](int angle) { /* ... */ }, [](int angle) { /* ... */ });
```

Naming the type of the variable the result is assigned to does not supply it – template arguments are deduced from the call arguments alone, never from what the returned value is assigned to:

```
// ill formed: "no matching function for call to connect"
untangle::actuator<std::function<void(int)>> actuator_rotate =
    untangle::connect([](int angle) { /* ... */ });
```

One named action anywhere in the call deduces the type for the whole of it, and the anonymous lambdas beside it then need nothing:

```
auto actuator_rotate = untangle::connect(action1, [](int angle) { /* ... */ });
```

The named form of `connect()` works the same way: a pair holding a pointer names an action the caller owns, a pair holding the action by value hands it over to the actuator, and the two can be mixed. The action type is deduced from the pointer, so a call whose first pair holds a lambda has to name the signature:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    std::make_pair("triangle", [](int angle) { /* ... */ }),
    std::make_pair("circle", [](int angle) { /* ... */ }));

// deduced from the leading pointer, so the lambda beside it needs nothing
auto actuator_mixed = untangle::connect(std::make_pair("triangle", &action1),
    std::make_pair("circle", [](int angle) { /* ... */ }));
```

For convenience there are provided helpers methods to "connect" to an initial list of "actions", or to create bindings to class methods.

Please check the manual in `doc/refman.pdf` for further references.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [11](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::actuator functor	17
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::actuator< action_t >	
An actuator is a functor that can trigger a dynamic list of actions (of type std::function<...>)	21
untangle::invalid_action	
Invalid action exception	32

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

actuator.hpp		35
------------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Functions

- `template<typename action_t, typename... Actions>`
`actuator< action_t > untangle::connect (action_t &A1, Actions &&... An)`
Creates an actuator holding an initial list of actions.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > untangle::connect (std::type_identity_t< action_t > &&A1, Actions &&... An)`
Creates an actuator owning an initial list of actions.
- `template<typename action_t, typename key_t, typename... Actions>`
`actuator< action_t > untangle::connect (std::pair< key_t, action_t * > A1, Actions &&... An)`
Creates an actuator holding an initial map of named actions.
- `template<typename action_t, typename key_t, typename value_t, typename... Actions>`
requires (!std::is_same_v<value_t, action_t*>)
`actuator< action_t > untangle::connect (std::pair< key_t, value_t > A1, Actions &&... An)`
Creates an actuator owning an initial map of named actions.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵ T>::type>>`
`action_t untangle::bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵ T>::type>>`
`action_t untangle::bind (class_t *obj, T class_t::*method)`
Binding to a class method.

6.1.1 Detailed Description

6.1.2 Function Documentation

6.1.2.1 connect() [1/4]

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    action_t & A1,
    Actions &&... An)
```

Creates an actuator holding an initial list of actions.

Remarks

Ownership: an action passed as an lvalue is one the caller owns, and the actuator only points at it – it must outlive the actuator. An action passed as an rvalue, an anonymous lambda above all, is moved into the actuator instead and needs no named variable at all. The two can be mixed in one call. See the ownership convention on [actuator](#).

Warning

A call made only of anonymous lambdas can not deduce `action_t`. A lambda has its own closure type and is not a `std::function` until something converts it, so there is nothing in such a call to deduce the action signature from, and it has to be named:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    [](int angle) { ... }, [](int angle) { ... });
```

Warning

Naming the type of the variable the result is assigned to does not supply it:

```
// still ill formed -- "no matching function for call to connect"
untangle::actuator<std::function<void(int)>> actuator_rotate =
    untangle::connect([](int angle) { ... });
```

Template arguments are deduced from the call arguments alone, never from what the returned value is assigned to, so the explicit argument belongs on [connect\(\)](#) itself. One named action anywhere in the call deduces `action_t` for the whole of it, and the anonymous lambdas beside it then need nothing.

Parameters

A1	- The first action. It is specified as <code>std::function<...></code> .
An	- Any number of further actions, each convertible to the type of A1.

Returns

An [actuator](#).

Example:

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle) {
    for (const auto& s : shapes) {
        s->rotate(angle);
    }
}

auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);

// connect() deduces the action type from its arguments. A lambda has its own closure type
// and is not a std::function until something converts it, so a call made only of anonymous
// lambdas has nothing to deduce from and the signature has to be named on connect()
// itself. Naming the type of the variable assigned to would not supply it: template
// arguments are never deduced from what the returned value is assigned to.
auto actuator_scale = untangle::connect<std::function<int(int)>>([](int v) { return v * 2; },
    [](int v) { return v * 3; });

actuator_scale(10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20, 30));
```

6.1.2.2 `connect()` [2/4]

```
template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    std::type_identity_t< action_t > && A1,
    Actions &&... An)
```

Creates an actuator owning an initial list of actions.

This overload takes the first action by rvalue, so it is the one that accepts a leading anonymous lambda. `std::type_identity_t` makes `action_t` a non-deduced context here, which is what both keeps this overload out of the way of the lvalue one – it is not a candidate at all unless `action_t` is named – and states the requirement described by the warning on the overload taking the first action by lvalue.

Parameters

A1	- The first action, as an rvalue.
An	- Any number of further actions, owned or pointed at by value category.

Returns

An **actuator**.

6.1.2.3 `connect()` [3/4]

```
template<typename action_t, typename key_t, typename... Actions>
actuator< action_t > untangle::connect (
    std::pair< key_t, action_t * > A1,
    Actions &&... An)
```

Creates an actuator holding an initial map of named actions.

Remarks

Ownership works as it does for the unnamed **connect()** overloads: a pair holding a pointer names an action the caller owns, and a pair holding an action by value – an anonymous lambda above all – hands it to the actuator to own. The two can be mixed in one call.

Warning

The action type is deduced from the pointer in the first pair, so a call whose first pair holds an anonymous lambda has nothing to deduce it from and has to name the signature on **connect()** itself, exactly as the unnamed overload does:

```
auto actuator_rotate = untangle::connect<std::function<void(int)>>(
    std::make_pair("triangle", [](int angle) { ... })),
    std::make_pair("circle", [](int angle) { ... }));
```

Remarks

`action_t` leads the template parameter list, ahead of `key_t`, so that naming it – **connect**<std::function<...>>(...) – means the same thing on every **connect()** overload. Both parameters are deduced from the first pair whenever it holds a pointer, so nothing has to be named here in the first place.

Parameters

A1	- The first name/action pair. Its second element is a pointer to an action.
An	- Any number of further name/action pairs, owned or pointed at according to whether they hold the action itself or a pointer to it.

Returns

An [actuator](#).

Example:

```
// A pair holding the action itself hands it to the actuator to own. The action type is
// deduced from a pointer, and there is no pointer here, so the signature has to be named
// on connect() -- as it does for the unnamed overload.
auto actuator_scale = untangle::connect<std::function<int(int)>>(
    std::make_pair("double", [](int v) { return v * 2; }),
    std::make_pair("triple", [](int v) { return v * 3; }));

actuator_scale.invoke_action("double", 10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20));
actuator_scale.invoke_action("triple", 10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(30));
```

6.1.2.4 connect() [4/4]

```
template<typename action_t, typename key_t, typename value_t, typename... Actions>
requires (!std::is_same_v<value_t, action_t*>)
actuator< action_t > untangle::connect (
    std::pair< key_t, value_t > A1,
    Actions &&... An)
```

Creates an actuator owning an initial map of named actions.

This overload takes a first pair holding the action itself rather than a pointer to it, so it is the one that accepts a leading anonymous lambda. `action_t` is never deduced here – the `requires` clause is what keeps it from competing with the overload taking a pointer in the first pair over a leading pointer pair, which both would otherwise accept once `action_t` is named.

Parameters

A1	- The first name/action pair, holding the action itself.
An	- Any number of further name/action pairs.

Returns

An [actuator](#).

6.1.2.5 bind() [1/2]

```
template<typename class__t, typename T, typename action__t = std::function<typename function_remove_const<T>::<
type>>
action__t untangle::bind (
    const std::shared_ptr< class__t > & obj,
    T class__t::* method)
```

Binding to a class function member.

It returns a `std::function(lambda)` that wraps the function member. It may be used to provide an action for [connect\(\)](#) or [actuator::add\(\)](#).

Remarks

It requires a shared pointer to the class type. A `std::weak_ptr` to it is captured internally in a lambda, so the binding can always check whether the shared object is still alive without keeping it alive itself. Therefore, it is safe to use actions provided by this binding inside an [actuator](#), and safe for the action to outlive the caller's shared pointer.

Parameters

obj	- Class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>

Returns

action__t - A `std::function` that wraps the pointer to function member.

Remarks

If the class object gets invalid, invoking this binding will throw an exception of type [invalid_action](#).

6.1.2.6 bind() [2/2]

```
template<typename class__t, typename T, typename action__t = std::function<typename function_remove_const<T>::<
type>>
action__t untangle::bind (
    class__t * obj,
    T class__t::* method)
```

Binding to a class method.

Attention

It is not safe to use this binding when the pointed-to object may be destroyed. A null pointer is detected and reported as a dead action, but a pointer to an already destroyed object can not be distinguished from a valid one. Prefer the overload taking a `std::shared_ptr`.

Remarks

It is provided for convenience of use: within a class it is safe to create bindings through this pointer.

Binding a null pointer produces an action that is always dead: invoking it throws [invalid_action](#).

Parameters

obj	- Pointer to class.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code>

Returns

action_t - A `std::function` that wraps the pointer to function member.

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::actuator](#) functor.

Classes

- struct [invalid_action](#)
Invalid action exception.
- struct [actuator](#)
An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Functions

- `template<typename... Args>`
`auto last\_arg (Args &&... args)`
Get last argument of params pack;.
- `template<typename action_t, typename arg_t>`
`void connect\_one (actuator< action_t > &target, arg_t &&arg)`
Add one [connect\(\)](#) argument to the actuator being built.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > connect (action_t &A1, Actions &&... An)`
Creates an actuator holding an initial list of actions.
- `template<typename action_t, typename... Actions>`
`actuator< action_t > connect (std::type_identity_t< action_t > &&A1, Actions &&... An)`
Creates an actuator owning an initial list of actions.
- `template<typename action_t, typename entry_t>`
`void connect\_named\_one (actuator< action_t > &target, entry_t &&entry)`
Add one named [connect\(\)](#) argument to the actuator being built.
- `template<typename action_t, typename key_t, typename... Actions>`
`actuator< action_t > connect (std::pair< key_t, action_t * > A1, Actions &&... An)`
Creates an actuator holding an initial map of named actions.
- `template<typename action_t, typename key_t, typename value_t, typename... Actions>`
`requires (!std::is_same_v<value_t, action_t*>)`
`actuator< action_t > connect (std::pair< key_t, value_t > A1, Actions &&... An)`
Creates an actuator owning an initial map of named actions.

- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵
T>::type>>
action_t bind (const std::shared_ptr< class_t > &obj, T class_t::*method)`
Binding to a class function member.
- `template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↵
T>::type>>
action_t bind (class_t *obj, T class_t::*method)`
Binding to a class method.

7.1.1 Detailed Description

Interface to `untangle::actuator` functor.

7.1.2 Function Documentation

7.1.2.1 last_arg()

```
template<typename... Args>
auto untangle::last_arg (
    Args &&... args)
```

Get last argument of params pack;.

Template Parameters

Args	params template types
------	-----------------------

Parameters

args	params
------	--------

Returns

last param value in case it exist, otherwise 0.

7.1.2.2 connect_one()

```
template<typename action_t, typename arg_t>
void untangle::connect_one (
    actuator< action_t > & target,
    arg_t && arg)
```

Add one `connect()` argument to the actuator being built.

An argument that is an lvalue of the action type is one the caller owns: only its address is stored, and it has to outlive the actuator. Anything else – an anonymous lambda, a temporary `std::function`, a named lambda that is not an `action_t` yet – has no owner to outlive the actuator, so it is converted to an action and moved into it.

Template Parameters

action↔ _t	Action type of the actuator being built.
arg_t	Deduced type of the argument, carrying its value category.

Parameters

target	- The actuator being built.
arg	- One connect() argument.

7.1.2.3 connect_named_one()

```
template<typename action_t, typename entry_t>
void untangle::connect_named_one (
    actuator< action_t > & target,
    entry_t && entry)
```

Add one named [connect\(\)](#) argument to the actuator being built.

It is [connect_one\(\)](#) for the named overloads: the second element of the pair decides ownership. A pointer to an action is one the caller owns and is only stored; anything else – an anonymous lambda, a temporary `std::function` – is converted to an action and moved into the actuator.

Remarks

A null pointer is dropped rather than stored, which is what the named [connect\(\)](#) has always done with one. An owned action that is empty is dropped for the same reason the unnamed [connect_one\(\)](#) drops one: nothing would ever point at it.

Template Parameters

action↔ _t	Action type of the actuator being built.
entry_t	Deduced type of the pair, carrying its value category.

Parameters

target	- The actuator being built.
entry	- One name/action pair.

Chapter 8

Class Documentation

8.1 untangle::actuator< action_t > Struct Template Reference

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

```
#include <actuator.hpp>
```

Public Types

- using `actions_t` = `std::list<action_t*>`
Actions container type.
- using `results_t` = `std::vector<typename result_t::type>`
Results container type.
- using `errors_t` = `std::vector<std::exception_ptr>`
Errors container type.

Public Member Functions

- `actuator` (const actuator &other)
Copy constructor.
- actuator & `operator=` (const actuator &other)
Assignment operator.
- action_t `type` () const
Helper method to access the action type of this object.
- void `reset` ()
Remove all actions, and any stored results and errors.
- void `copy_from` (const actuator &other)
Copy another actuator, re-pointing the copied actions at this object's storage.
- void `release_owned` (const action_t *action)
Destroy an owned action that nothing points at any more.
- template<typename last_t>
void `invoke_callback` (last_t &last)
Invoke the trailing callback argument, if the invocation has one.
- template<typename... Args>
void `operator()` (Args &&... args)

- The call operator.

 - `template<typename... Args>`
`void invoke\_action (const std::string &name, Args &&... args)`
 Invokes one single action associated with a key.
 - `void add (action__t *action)`
 Add an action to the actions list.
 - `action__t * add (action__t &&action)`
 Add an action the actuator owns.
 - `void add (const std::string &name, action__t *action)`
 Add action to the actions map associated with a name.
 - `action__t * add (const std::string &name, action__t &&action)`
 Add an action the actuator owns, associated with a name.
 - `void remove (const action__t *action)`
 Remove an action from the actions list.
 - `void remove (const std::string &name)`
 Remove an action from actions map.
 - `bool is\_connected () const`
 Check if this actuator is "connected" with other actions.
 - `bool has\_action (const std::string &name) const`
 Check if there is certain named action.

Public Attributes

- `actions\_\_t actions`
 Actions list.
- `actions\_map\_\_t actions_map`
 Named actions map.
- `results\_\_t results`
 Actions return values list.
- `errors\_\_t errors`
 What the actions threw. See [errors__t](#).
- `std::list< action__t > owned`
 Actions this actuator owns, as added by [add\(action__t&&\)](#).

8.1.1 Detailed Description

```
template<typename action__t>
struct untangle::actuator< action__t >
```

An actuator is a functor that can trigger a dynamic list of actions (of type `std::function<...>`).

Remarks

An actuator object can be constructed with an initial list of actions by [connect\(\)](#).

Callback convention: when the last argument of an invocation can be called with the action return type `R` and returns nothing, it is not passed on as a plain argument alone – it is also treated as a completion callback. The action is invoked with the full argument list as usual, and the callback is then invoked with the action return value. Any void returning callable qualifies: a `std::function<void(R)>`, a lambda taking `R`, a function pointer. It applies to [operator\(\)\(\)](#) and to [invoke_action\(\)](#) alike. Actions returning void have no result to report, so no callback is invoked for them.

Warning

The return type is what separates a callback from an argument the action means to consume itself. A trailing callable that returns a value – a transform, a comparator – is left alone, and by the same rule a callback written to return something is silently not invoked. A callback returns nothing.

Remarks

Argument convention: an action must not take ownership of the arguments it is invoked with. `operator()` forwards one argument pack to every action in the list, and an rvalue can be moved from only once, so an action that consumes an argument leaves the actions after it holding a moved-from object – an empty `std::function` for a callback a later action means to call itself. The arguments are forwarded rather than copied so that a single action pays nothing for the broadcast; what that buys has to be respected by the actions.

Warning

An action does not have to move an argument in its body to consume it: a parameter taken by value is move constructed from an rvalue by the call itself. Take arguments by reference or const reference, or invoke with lvalues, whenever more than one action is connected. The behaviour is otherwise unpredictable, and it is the caller and the action signatures together that decide it – an actuator cannot detect the violation.

Remarks

`invoke_action()` invokes one single action, so nothing follows it and the convention does not constrain it.

Failure convention: an action that throws does not stop the invocation. What it threw is stored in `actuator::errors`, one `std::exception_ptr` per failure, and the actions after it still run. An `untangle::invalid_action` means a dead binding, so that action is dropped as well; anything else leaves the action in place. Nothing is printed and nothing escapes the call - the caller reads `actuator::errors` and decides.

Ownership convention: an actuator normally does not own its actions. It stores pointers to `std::function` objects the caller keeps alive, and those have to outlive it. `add(action_t&&)` is the exception – it moves the action into `actuator::owned` and points at the stored copy – which is what lets an anonymous lambda be an action, with no named variable to keep around. The handle it returns is the only way to `remove()` such an action afterwards.

Template Parameters

<code>action_t</code>	Action type. It is specified as <code>std::function<...></code> .
-----------------------	---

8.1.2 Member Typedef Documentation

8.1.2.1 actions_t

```
template<typename action_t>
using untangle::actuator< action_t >::actions_t = std::list<action_t*>
```

Actions container type.

Remarks

The elements stored are of pointer type, that is required to implement the `remove()` operation. `std::function` supports only equality operator for nullptr (two `std::function(s)` can not compare).

8.1.2.2 results_t

```
template<typename action_t>
using untangle::actuator< action_t >::results_t = std::vector<typename result_t::type>
```

Results container type.

It holds the return values of the actions that have a non-void return type. Upon the actuator invocation, the returns can be extracted from [results](#).

8.1.2.3 errors_t

```
template<typename action_t>
using untangle::actuator< action_t >::errors_t = std::vector<std::exception_ptr>
```

Errors container type.

It holds what the actions threw, as `std::exception_ptr`, in the order they were invoked.

8.1.3 Constructor & Destructor Documentation

8.1.3.1 actuator()

```
template<typename action_t>
untangle::actuator< action_t >::actuator (
    const actuator< action_t > & other) [inline]
```

Copy constructor.

Remarks

It can not be defaulted, see [copy_from\(\)](#).

8.1.4 Member Function Documentation

8.1.4.1 operator=()

```
template<typename action_t>
actuator & untangle::actuator< action_t >::operator= (
    const actuator< action_t > & other) [inline]
```

Assignment operator.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
untangle::actuator<decltype(actuator_rotate.type())> actuator_rotate_1 = actuator_rotate;
actuator_rotate_1(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.4.2 type()

```
template<typename action_t>
action_t untangle::actuator< action_t >::type () const [inline]
```

Helper method to access the action type of this object.

Note

Intended to be used in expressions by decltype(<actuator instance>.type())

Returns

action_t

8.1.4.3 reset()

```
template<typename action_t>
void untangle::actuator< action_t >::reset () [inline]
```

Remove all actions, and any stored results and errors.

After this call the actuator is empty: `actuator::is_connected()` returns false. The actions it owns are destroyed, so every handle returned by `add(action_t&&)` is dangling afterwards; the actions it merely points at are left untouched.

8.1.4.4 copy_from()

```
template<typename action_t>
void untangle::actuator< action_t >::copy_from (
    const actuator< action_t > & other) [inline]
```

Copy another actuator, re-pointing the copied actions at this object's storage.

Remarks

A defaulted copy is wrong as soon as an actuator owns anything: it duplicates `actuator::owned` but leaves `actuator::actions` pointing into other's copy of it, so the new object dangles the moment other is destroyed. The owned actions are copied first, then each pointer into other::owned is translated to the matching element here.

A pointer to an action the caller owns is external to both actuators and is copied unchanged: it is not in either `actuator::owned`, so it is not in the translation table and is left as it is.

Parameters

other	- The actuator to copy. Self copy is the caller's to exclude.
-------	---

8.1.4.5 release_owned()

```
template<typename action_t>
void untangle::actuator< action_t >::release_owned (
    const action_t * action) [inline]
```

Destroy an owned action that nothing points at any more.

Remarks

Removing an action only drops a pointer to it. When the actuator owns that action the `std::function` itself lives in `actuator::owned` and has to go too, or a loop of `add(action_t&&)` and `remove()` grows `actuator::owned` without bound.

The action is kept while anything still refers to it: one handle can be added to both the list and the map, and removing it from one must not leave the other pointing at a destroyed object. An action the actuator does not own is not in `actuator::owned`, so nothing is found to erase and the caller's object is left alone.

Parameters

action	- The action just removed. A null pointer is accepted and matches nothing.
--------	--

8.1.4.6 invoke_callback()

```
template<typename action_t>
template<typename last_t>
void untangle::actuator< action_t >::invoke_callback (
    last_t & last) [inline]
```

Invoke the trailing callback argument, if the invocation has one.

Does nothing unless `last_t` can be called with the action return type and returns nothing, so any other trailing argument is left alone. The test is on what the type can do, not on what it is, so a plain lambda qualifies as well as a `std::function`. See the callback convention on [actuator](#).

Remarks

The single condition does the work of two tests: (1) whether `last` can be called with the action return type at all, and (2) whether that call returns nothing. Spelled out as traits they need two nested `if constexpr`:

```
// (1)
if constexpr (std::is_invocable_v<last_t&, typename result_t::type>) {
    // (2)
    if constexpr (std::is_void_v<std::invoke_result_t<last_t&, typename result_t::type>)>) {
        last(results.back());
    }
}
```

Both are folded into the one expression instead, each in a different part of it:

```
requires { requires std::is_void_v<decltype( last(results.back()) )>; }
// \underline{(2)} \underline{(1)}
```

Remarks

(1) Invocability comes for free. To form `decltype(last(results.back()))` the compiler must first form the call `last(...)`. If `last` is an int, that expression does not exist – and inside a `requires-expression` a substitution failure yields `false` instead of being ill formed, the same property concepts are built on. So invocability is never tested separately: it is a precondition of the return type question, and failing it is what makes the whole condition false.

(2) The return type test is the inner `requires`, and the doubled keyword is not a typo. `requires { expr; }` asks only whether `expr` is well formed, and `std::is_void_v<...>` is well formed when it is `false` too – that spelling would let a value returning callable through. The second keyword makes it a nested requirement: a condition that has to hold, not merely compile.

The trait spelling has to nest, which is part of why it is not the one used here: the two `if constexpr` cannot be joined with `&&`. `&&` short circuits evaluation, not instantiation, so `std::invoke_result_t` is formed whatever the first trait says, and for a type that is not invocable at all it has no `::type` – a hard error rather than a false result.

The caller must copy the callback out of the argument pack before invoking the action: an action taking it by value moves from the caller's `std::function`, which would leave this an empty function to call.

Template Parameters

<code>last_</code> <code>_t</code>	Type of the last argument, as returned by <code>last_arg()</code> .
---------------------------------------	---

Parameters

<code>last</code>	- Last argument value, copied before the action was invoked.
-------------------	--

8.1.4.7 operator>()()

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::operator() (
    Args &&... args) [inline]
```

The call operator.

Actions in the `actuator::actions` list are triggered by invoking the call operator.

Parameters

<code>args</code>	- Arguments list must match the action arity. A trailing callback is invoked once per action, with that action's return value; see the convention on <code>actuator</code> .
-------------------	--

Warning

The same argument pack is forwarded to every action in the list, so the argument convention on [actuator](#) applies here in full: an action that consumes an argument leaves the actions after it with a moved-from object.

Remarks

Nothing escapes this call: what an action throws goes to [actuator::errors](#), and the actions after it still run. See the failure convention on [actuator](#).

8.1.4.8 `invoke_action()`

```
template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::invoke_action (
    const std::string & name,
    Args &&... args) [inline]
```

Invokes one single action associated with a key.

An action that can not be invoked – a null pointer, an empty `std::function`, a binding whose object is gone – is dropped from [actuator::actions_map](#) instead, exactly as [operator\(\)\(\)](#) drops one from [actuator::actions](#). Nothing is invoked and nothing escapes; [actuator::has_action\(\)](#) reports it gone afterwards.

Remarks

What the action throws is stored in [actuator::errors](#), as it is for [operator\(\)\(\)](#). See the failure convention on [actuator](#).

Parameters

name	- Key associated with the action. Invoking a key that is not in the map does nothing.
args	- Arguments list must match the action arity. A trailing callback is invoked with the action's return value; see the convention on actuator .

8.1.4.9 `add()` [1/4]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    action_t * action) [inline]
```

Add an action to the actions list.

Parameters

action	- Action to be added.
--------	-----------------------

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(2).WillRepeatedly(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.add(&action1);
actuator_rotate(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.4.10 add() [2/4]

```
template<typename action_t>
action_t * untangle::actuator< action_t >::add (
    action_t && action) [inline]
```

Add an action the actuator owns.

The action is moved into [actuator::owned](#), so it needs no named variable outliving the actuator: an anonymous lambda can be passed straight in, and the call converts it to `action_t`. See the ownership convention on [actuator](#).

Parameters

action	- Action to be added. It is taken by value and moved from.
--------	--

Returns

`action_t*` - A handle to the stored action, to pass to [remove\(\)](#). It stays valid until that action is removed or the actuator is destroyed. A copy of the actuator owns its own copy of the action and has its own handle to it.

Example:

```
// An anonymous lambda has no named variable to outlive the actuator, so the actuator takes
// ownership of it: it is moved into actuator::owned and the list points at the stored copy.
untangle::actuator<std::function<int(int)>> actuator_scale;
actuator_scale.add([](int v) { return v * 2; });
actuator_scale.add([](int v) { return v * 3; });

actuator_scale(10);
ASSERT_THAT(actuator_scale.results, ::testing::ElementsAre(20, 30));
```

8.1.4.11 add() [3/4]

```
template<typename action_t>
void untangle::actuator< action_t >::add (
    const std::string & name,
    action_t * action) [inline]
```

Add action to the actions map associated with a name.

Parameters

name	- Name of the action.
action	- Action to be added.

8.1.4.12 add() [4/4]

```
template<typename action_t>
action_t * untangle::actuator< action_t >::add (
    const std::string & name,
    action_t && action) [inline]
```

Add an action the actuator owns, associated with a name.

Parameters

name	- Name of the action.
action	- Action to be added. It is taken by value and moved from.

Returns

action_t* - A handle to the stored action, as [add\(action_t&&\)](#) returns, or nullptr when name is already taken. The name already in the map keeps the action it has, as it does for [add\(const std::string&, action_t*\)](#), and the action offered here is dropped rather than left ownerless in [actuator::owned](#).

8.1.4.13 remove() [1/2]

```
template<typename action_t>
void untangle::actuator< action_t >::remove (
    const action_t * action) [inline]
```

Remove an action from the actions list.

An invalid action (empty std::function) is implicitly removed when [operator\(\)\(\)](#) is invoked.

An action the actuator owns is destroyed by this call, so the handle to it is dangling afterwards. See [release_owned\(\)](#).

Parameters

action	- Action to be removed. For an owned action this is the handle returned by add(action_t&&) .
--------	--

Example:

```
const auto t = std::make_shared<triangle_mock>();
```

```

const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(0);
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.remove(&action1);
actuator_rotate(50);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());

```

8.1.4.14 remove() [2/2]

```

template<typename action_t>
void untangle::actuator< action_t >::remove (
    const std::string & name) [inline]

```

Remove an action from actions map.

An action the actuator owns is destroyed by this call, so the handle to it is dangling afterwards. See [release_owned\(\)](#).

Parameters

name	- Name of the action to remove.
------	---------------------------------

8.1.4.15 is_connected()

```

template<typename action_t>
bool untangle::actuator< action_t >::is_connected () const [inline]

```

Check if this actuator is "connected" with other actions.

Returns

- true - if the [actuator::actions](#) list is not empty.
- false - if the [actuator::actions](#) list is empty.

8.1.4.16 has_action()

```

template<typename action_t>
bool untangle::actuator< action_t >::has_action (
    const std::string & name) const [inline]

```

Check if there is certain named action.

Parameters

name	- Name associated with the action.
------	------------------------------------

Returns

- true - if name can be found in [actuator::actions_map](#)
- false - if name can not be found in [actuator::actions_map](#)

8.1.5 Member Data Documentation

8.1.5.1 owned

```
template<typename action_t>
std::list<action_t> untangle::actuator< action_t >::owned
```

Actions this actuator owns, as added by [add\(action_t&&\)](#).

Remarks

It is a `std::list` because inserting into one never invalidates the address of an element already in it, so every handle already handed out stays valid as further actions are added. A `std::vector` would reallocate and dangle all of them at once.

The documentation for this struct was generated from the following file:

- [actuator.hpp](#)

8.2 [untangle::invalid_action](#) Struct Reference

Invalid action exception.

```
#include <actuator.hpp>
```

Public Member Functions

- [invalid_action](#) (std::string text)
Construct a new invalid action object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.2.1 Detailed Description

Invalid action exception.

Remarks

An action may be provided as a binding to a class function member, by using [untangle::bind\(\)](#). When the class object gets invalid, invoking the action will raise an exception to this type.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.2.3 Member Function Documentation

8.2.3.1 what()

`const char * untangle::invalid_action::what () const` [inline], [override], [noexcept]

The message text describing the reason of this exception.

Returns

`const char*` - The message text.

The documentation for this struct was generated from the following file:

- actuator.hpp

Chapter 9

File Documentation

9.1 actuator.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <exception>
00009 #include <functional>
00010 #include <list>
00011 #include <map>
00012 #include <memory>
00013 #include <string>
00014 #include <type_traits>
00015 #include <utility>
00016 #include <vector>
00017
00018 namespace untangle {
00019 // exception
00028 struct invalid_action : std::exception {
00034     explicit invalid_action(std::string text) : message(std::move(text)) {}
00035
00041     const char* what() const noexcept override { return message.c_str(); }
00042
00043 private:
00044     std::string message;
00045 };
00046
00054 template <typename... Args>
00055 auto last_arg(Args&&... args) {
00056     if constexpr (sizeof...(Args) <= 0) {
00057         return 0;
00058     } else {
00059         return std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00060     }
00061 }
00062
00114 template <typename action_t>
00115 struct actuator final {
00123     using actions_t = std::list<action_t*>;
00124     using actions_map_t = std::map<std::string, action_t*>;
00125     using result_t = std::conditional<std::is_void<typename action_t::result_type>::value, int,
00126                                     typename action_t::result_type>;
00133     using results_t = std::vector<typename result_t::type>;
00139     using errors_t = std::vector<std::exception_ptr>;
00140
00141     actions_t actions;
00142     actions_map_t actions_map;
00143     results_t results;
00144     errors_t errors;
00145
00153     std::list<action_t> owned;
00154
00155     actuator() = default;
00161     actuator(const actuator& other) { copy_from(other); }
00162     actuator(actuator&&) noexcept = default;
00163     ~actuator() = default;
00170     actuator& operator=(const actuator& other) {
00171         if (this != &other) {
00172             copy_from(other);

```

```

00173     }
00174     return *this;
00175 }
00176 actuator& operator=(actuator&&) noexcept = default;
00177
00185 action_t type() const { return nullptr; }
00186
00194 void reset() {
00195     actions.clear();
00196     actions_map.clear();
00197     results.clear();
00198     errors.clear();
00199     owned.clear();
00200 }
00201
00216 void copy_from(const actuator& other) {
00217     owned = other.owned;
00218
00219     // Old address -> new address, for the owned actions only.
00220     std::map<const action_t*, action_t*> remap;
00221     auto src = other.owned.begin();
00222     auto dst = owned.begin();
00223     for (; src != other.owned.end(); ++src, ++dst) {
00224         remap.emplace(&*src, &*dst);
00225     }
00226     const auto translate = [&remap](action_t* action) {
00227         const auto it = remap.find(action);
00228         return it != remap.end() ? it->second : action;
00229     };
00230
00231     actions.clear();
00232     for (const auto& action : other.actions) {
00233         actions.push_back(translate(action));
00234     }
00235     actions_map.clear();
00236     for (const auto& entry : other.actions_map) {
00237         actions_map.emplace(entry.first, translate(entry.second));
00238     }
00239     results = other.results;
00240     errors = other.errors;
00241 }
00242
00257 void release_owned(const action_t* action) {
00258     for (const auto& a : actions) {
00259         if (a == action) {
00260             return;
00261         }
00262     }
00263     for (const auto& entry : actions_map) {
00264         if (entry.second == action) {
00265             return;
00266         }
00267     }
00268     owned.remove_if([&action](const action_t& a) { return &a == action; });
00269 }
00270
00326 template <typename last_t>
00327 void invoke_callback(last_t& last) {
00328     if constexpr (requires { requires std::is_void_v<decltype(last(results.back()))>; }) {
00329         last(results.back());
00330     }
00331 }
00332
00348 template <typename... Args>
00349 void operator()(Args&&... args) {
00350     results.clear();
00351     errors.clear();
00352
00353     // Copied up front, before any action can move the callback out of the argument pack.
00354     auto last = last_arg(args...);
00355
00356     // Dead bindings are collected here and dropped after the loop.
00357     // The actuator does not own the actions it points at, so it must never
00358     // write through action_t* into a std::function belonging to the caller.
00359     std::vector<action_t*> dead_actions;
00360
00361     for (const auto& action : actions) {
00362         // A null pointer or an empty std::function can never be invoked. Calling an
00363         // empty one throws std::bad_function_call, which is not an invalid_action and
00364         // would escape this operator, so drop it instead of invoking it.
00365         if (action == nullptr || !*action) {
00366             dead_actions.push_back(action);
00367             continue;
00368         }
00369
00370         try {
00371             if constexpr (std::is_same_v<typename action_t::result_type, void>) {

```

```

00372     (*action)(std::forward<Args>(args)...);
00373 } else {
00374     results.push_back((*action)(std::forward<Args>(args)...));
00375     invoke_callback(last);
00376 }
00377 } catch (const invalid_action&) {
00378     // A dead binding is not coming back: it is recorded and the action is dropped.
00379     errors.push_back(std::current_exception());
00380     dead_actions.push_back(action);
00381 } catch (...) {
00382     // Anything else is the action's own failure. It is recorded, the action is kept, and the
00383     // actions behind it still run.
00384     errors.push_back(std::current_exception());
00385 }
00386 }
00387
00388 for (const auto& dead_action : dead_actions) {
00389     actions.remove(dead_action);
00390     release_owned(dead_action);
00391 }
00392 }
00393
00400 template <typename... Args>
00401 void invoke_action(const std::string& name, Args&&... args) {
00402     results.clear();
00403     errors.clear();
00404
00405     // Copied up front, before the action can move the callback out of the argument pack.
00406     auto last = last_arg(args...);
00407     const auto it = actions_map.find(name);
00408     if (it == actions_map.end()) {
00409         return;
00410     }
00411
00412     // A null pointer or an empty std::function can never be invoked, and dereferencing a
00413     // null one to try is undefined behaviour before a call is even made. Calling an empty
00414     // one throws std::bad_function_call, which is not an invalid_action and would escape
00415     // this method. Drop it instead, as operator()() does for the actions list.
00416     if (it->second == nullptr || !*it->second) {
00417         const action_t* dead_action = it->second;
00418         actions_map.erase(it);
00419         release_owned(dead_action);
00420         return;
00421     }
00422
00423     try {
00424         if constexpr (std::is_same_v<typename action_t::result_type, void>) {
00425             (*it->second)(std::forward<Args>(args)...);
00426         } else {
00427             results.push_back((*it->second)(std::forward<Args>(args)...));
00428             invoke_callback(last);
00429         }
00430     } catch (const invalid_action&) {
00431         errors.push_back(std::current_exception());
00432         const action_t* dead_action = it->second;
00433         actions_map.erase(name);
00434         release_owned(dead_action);
00435     } catch (...) {
00436         errors.push_back(std::current_exception());
00437     }
00438 }
00439
00440 void add(action_t* action) { actions.push_back(action); }
00441
00442 action_t* add(action_t&& action) {
00443     owned.push_back(std::move(action));
00444     actions.push_back(&owned.back());
00445     return &owned.back();
00446 }
00447
00448 void add(const std::string& name, action_t* action) { actions_map.emplace(name, action); }
00449
00450 action_t* add(const std::string& name, action_t&& action) {
00451     owned.push_back(std::move(action));
00452     const auto [it, inserted] = actions_map.emplace(name, &owned.back());
00453     if (!inserted) {
00454         owned.pop_back();
00455         return nullptr;
00456     }
00457     return &owned.back();
00458 }
00459
00460 void remove(const action_t* action) {
00461     actions.remove_if([&action](const auto& a) { return (action == a); });
00462     release_owned(action);
00463 }
00464

```

```

00536 void remove(const std::string& name) {
00537     const auto it = actions_map.find(name);
00538     if (it == actions_map.end()) {
00539         return;
00540     }
00541     const action_t* action = it->second;
00542     actions_map.erase(it);
00543     release_owned(action);
00544 }
00545
00552 bool is_connected() const { return !actions.empty() || !actions_map.empty(); }
00553
00561 bool has_action(const std::string& name) const {
00562     return actions_map.find(name) != actions_map.end();
00563 }
00564 };
00565
00583 template <typename action_t, typename arg_t>
00584 void connect_one(actuator<action_t>& target, arg_t&& arg) {
00585     if constexpr (std::is_lvalue_reference_v<arg_t&&> &&
00586         std::is_same_v<std::decay_t<arg_t>, action_t>) {
00587         if (arg != nullptr) {
00588             target.add(&arg);
00589         }
00590     } else {
00591         action_t action(std::forward<arg_t>(arg));
00592         if (action != nullptr) {
00593             target.add(std::move(action));
00594         }
00595     }
00596 }
00597
00640 template <typename action_t, typename... Actions>
00641 actuator<action_t> connect(action_t& A1, Actions&&... An) {
00642     actuator<action_t> target;
00643     connect_one(target, A1);
00644     (connect_one(target, std::forward<Actions>(An)), ...);
00645     return target;
00646 }
00647
00664 template <typename action_t, typename... Actions>
00665 actuator<action_t> connect(std::type_identity_t<action_t>&& A1, Actions&&... An) {
00666     actuator<action_t> target;
00667     connect_one(target, std::move(A1));
00668     (connect_one(target, std::forward<Actions>(An)), ...);
00669     return target;
00670 }
00671
00672 template <typename actuator_t>
00673 void remove_empty_actions(actuator_t& actuator) {
00674     std::vector<typename actuator_t::actions_map_t::const_iterator> removable_iterators;
00675     for (auto it = actuator.actions_map.begin(); it != actuator.actions_map.end(); ++it) {
00676         if (it->second == nullptr) {
00677             removable_iterators.push_back(it);
00678         }
00679     }
00680     for (auto it : removable_iterators) {
00681         actuator.actions_map.erase(it);
00682     }
00683 }
00684
00702 template <typename action_t, typename entry_t>
00703 void connect_named_one(actuator<action_t>& target, entry_t&& entry) {
00704     if constexpr (std::is_same_v<std::remove_cvref_t<decltype(entry.second)>, action_t*>) {
00705         if (entry.second != nullptr) {
00706             target.add(entry.first, entry.second);
00707         }
00708     } else {
00709         action_t action(std::forward<entry_t>(entry).second);
00710         if (action != nullptr) {
00711             target.add(entry.first, std::move(action));
00712         }
00713     }
00714 }
00715
00750 template <typename action_t, typename key_t, typename... Actions>
00751 actuator<action_t> connect(std::pair<key_t, action_t*> A1, Actions&&... An) {
00752     actuator<action_t> target;
00753     connect_named_one(target, std::move(A1));
00754     (connect_named_one(target, std::forward<Actions>(An)), ...);
00755     return target;
00756 }
00757
00774 template <typename action_t, typename key_t, typename value_t, typename... Actions>
00775 requires(!std::is_same_v<value_t, action_t*>)
00776 actuator<action_t> connect(std::pair<key_t, value_t> A1, Actions&&... An) {
00777     actuator<action_t> target;

```

```

00778 connect_named_one(target, std::move(A1));
00779 (connect_named_one(target, std::forward<Actions>(An)), ...);
00780 return target;
00781 }
00782
00783 // generic helpers to remove const qualifier from a function type,
00784 // for instance const member functions
00785 template <typename T>
00786 struct function_remove_const;
00787
00788 template <typename R, typename... Args>
00789 struct function_remove_const<R(Args...)> {
00790     using type = R(Args...);
00791 };
00792
00793 template <typename R, typename... Args>
00794 struct function_remove_const<R(Args...) const> {
00795     using type = R(Args...);
00796 };
00797
00801
00823 template <typename class_t, typename T,
00824     typename action_t = std::function<typename function_remove_const<T>::type>
00825 action_t bind(const std::shared_ptr<class_t>& obj, T class_t::* method) {
00826     return [wp = std::weak_ptr<class_t>(obj), method](auto&&... args) ->
00827         typename action_t::result_type {
00828         // lock() also keeps the object alive for the duration of the call
00829         const auto obj_ = wp.lock();
00830         if (!obj_) {
00831             // inform the actuator about dead binding
00832             throw invalid_action("bind: invalid object");
00833         }
00834         return ((*obj_).*method)(std::forward<decltype(args)>(args)...);
00835     };
00836 }
00837
00858 template <typename class_t, typename T,
00859     typename action_t = std::function<typename function_remove_const<T>::type>
00860 action_t bind(class_t* obj, T class_t::* method) {
00861     return [obj, method](auto&&... args) -> typename action_t::result_type {
00862         if (!obj) {
00863             // inform the actuator about dead binding
00864             throw invalid_action("bind: invalid object");
00865         }
00866         return ((obj)->*method)(std::forward<decltype(args)>(args)...);
00867     };
00868 }
00869
00870 } // namespace untangle

```


Index

actions_t
 untangle::actuator< action_t >, 23
actuator, 1
 untangle::actuator< action_t >, 24
add
 untangle::actuator< action_t >, 28, 29, 30
bind
 namespace untangle: functions, 14, 15
connect
 namespace untangle: functions, 11, 12, 13, 14
connect_named_one
 untangle, 19
connect_one
 untangle, 18
copy_from
 untangle::actuator< action_t >, 25
errors_t
 untangle::actuator< action_t >, 24
has_action
 untangle::actuator< action_t >, 31
invalid_action
 untangle::invalid_action, 32
invoke_action
 untangle::actuator< action_t >, 28
invoke_callback
 untangle::actuator< action_t >, 26
is_connected
 untangle::actuator< action_t >, 31
last_arg
 untangle, 18
namespace untangle: functions, 11
 bind, 14, 15
 connect, 11, 12, 13, 14
operator()
 untangle::actuator< action_t >, 27
operator=
 untangle::actuator< action_t >, 24
owned
 untangle::actuator< action_t >, 32
release_owned
 untangle::actuator< action_t >, 25
remove
 untangle::actuator< action_t >, 30, 31
reset
 untangle::actuator< action_t >, 25
results_t
 untangle::actuator< action_t >, 23
type
 untangle::actuator< action_t >, 24
untangle, 17
 connect_named_one, 19
 connect_one, 18
 last_arg, 18
untangle::actuator< action_t >, 21
 actions_t, 23
 actuator, 24
 add, 28, 29, 30
 copy_from, 25
 errors_t, 24
 has_action, 31
 invoke_action, 28
 invoke_callback, 26
 is_connected, 31
 operator(), 27
 operator=, 24
 owned, 32
 release_owned, 25
 remove, 30, 31
 reset, 25
 results_t, 23
 type, 24
untangle::invalid_action, 32
 invalid_action, 32
 what, 33
what
 untangle::invalid_action, 33